

Parcial 1
LinguaPlus

Nombres
Jhon Edwin Pandales
Mariana Vásquez Zapata



Profesor: CAROLINA LONDOÑO IDARRAGA

Universidad del Quindío
Facultad de Ingeniería
Ingeniería de Sistemas y Computación
Programación 2
Armenia, septiembre 2026

Jhon Edwin Pandales
Mariana Vasquez Zapata

Programación 2
Parcial 1 - LinguaPlus

Video sustentación: https://youtu.be/0GOlw5MFK_c?si=t6vIxmcl-W-yVFNb

Pensamiento computacional:

Abstracción:

¿Qué se solicita finalmente? (problema)

Desarrollar un sistema de gestión para la academia LinguaPlus, con interfaz gráfica, que permita administrar estudiantes, docentes, programas de formación, matrículas y servicios adicionales, controlar las reglas de negocio asociadas y generar comprobantes de pago.

¿Qué información es relevante dado el problema anterior?

Usuario

- Identificación.
- Nombre completo.
- Teléfono.
- Correo electrónico.

La clase `Usuario` permite abstraer la información común de estudiantes y docentes.

Estudiante

- Edad.
- Fecha de registro.
- Información heredada de `Usuario`.
- Funcionalidad para determinar si su número telefónico es un número perfecto.

Docente

- Idioma de especialidad.
- Tarifa por sesión.
- Información heredada de `Usuario`.

Programa

- Código.
- Nombre.
- Idioma.
- Descripción.
- Duración en meses.
- Valor mensual.

- Estado.
- Modalidad.
- Número de cupos.

Los programas pueden ser:

- Programa básico.
- Programa intensivo.
- Programa personalizado.

El programa personalizado además maneja:

- Número de sesiones con tutor.
- Nivel de idioma.
- Objetivos del estudiante.

Matrícula

- Número de matrícula.
- Estudiante.
- Programa.
- Fecha de inicio.
- Docente tutor opcional.
- Servicios adicionales.
- Descuento.
- Observaciones.

Servicio adicional

- Código.
- Nombre.
- Descripción.
- Precio.
- Disponibilidad.

Periodo académico

- Nombre.
- Fecha de inicio.
- Fecha de finalización.
- Programas ofertados.

Modalidad

- Presencial.
- Virtual.

Insumos

Para la modalidad presencial se utilizan:

- Material impreso.
- Carné físico.

Para la modalidad virtual se utilizan:

- Licencia de plataforma.
- Carné digital.

Comprobantes

El sistema contempla diferentes formatos de comprobante:

- PDF.
- Excel.

La estructura permite incorporar posteriormente otros formatos.

¿Cómo se agrupa la información relevante?

Clase	Atributos
Usuario abstract	indentificacion : String nombreCompleto : String telefono : String correo : String
Estudiante	edad : int fechaRegistro : LocalDate Hereda: indentificacion, nombreCompleto, telefono, correo de Usuario
Docente	idiomaEspecialidad : String tarifaPorSeccion : double Hereda: indentificacion, nombreCompleto, telefono, correo de Usuario
Programa abstract	codigo : String nombre : String idioma : String descripcion : String duracionMeses : int valorMensual : double estado : Estado Programamodalidad : Modalidad

	cupos : int
ProgramaBasico	No define atributos propios.Hereda: todos los atributos de Programa
ProgramaIntensivo	No define atributos propios.Hereda: todos los atributos de Programa
ProgramaPersonalizado	sesionesTutor : int nivelIdioma : String objetivos : String Hereda: todos los atributos de Programa
Matricula	numero : long estudiante : Estudiante programa : Programa fechaInicio : LocalDate docenteTutor : Docente servicios : List<ServicioAdicional> descuento : double observaciones : String
ServicioAdicional	codigo : String nombre : String descripcion : String precio : double disponible : boolean
PeriodoAcademico	nombre : String fechaInicio : LocalDate fechaFin : LocalDate programasOfertados : List<Programa>
Academia	nombreComercial : String nit : String direccion : String telefono : String correo : String paginaWeb : String estudiantes : List<Estudiante> docentes : List<Docente> programasBase : List<Programa> periodos : List<PeriodoAcademico>

	matriculas : List<Matricula> serviciosAdicionales : List<ServicioAdicional>
--	--

Clases principales Clases e interfaces utilizadas para los patrones de diseño

Clase / interfaz	Atributos definidos en el código	Patrón / función
Matricula.Builder	estudiante : Estudiante programa : Programa fechaInicio : LocalDate docenteTutor : Docenteservicios : List<ServicioAdicional> descuento : double observaciones : String	Builder para construir Matricula
ConsecutivoMatricula	INSTANCIA : ConsecutivoMatricula static finalultimo : long	Singleton para generar números consecutivos
ProgramaFactory abstract	No define atributos	Factory Method para crear programas
BasicoFactory	No define atributos propios	Factory Method para crear ProgramaBasico
IntensivoFactory	No define atributos propios	Factory Method para crear ProgramaIntensivo
PersonalizadoFactory	sesionesTutor : int nivelIdioma : String objetivos : String	Factory Method para crear ProgramaPersonalizado
FabricaInsumos	No define atributos	Abstract Factory
FabricaPresencial	No define atributos	Abstract Factory para modalidad presencial
FabricaVirtual	No define atributos	Abstract Factory para modalidad virtual

GeneradorComprobant e abstract	No define atributos	Factory Method para generar comprobantes
GeneradorPDF	No define atributos	Factory Method para formato PDF
GeneradorExcel	No define atributos	Factory Method para formato Excel
FormatoComprobante	No define atributos	Interfaz para los formatos de comprobante
ComprobantePDF	No define atributos	Implementación del formato PDF
ComprobanteExcel	No define atributos	Implementación del formato Excel
ServicioEntregaInsumo s	No define atributos	Utiliza la fábrica de insumos

Interfaces y productos de la Abstract Factory

Interfaz / clase	Atributos definidos en el código	Función
Material	No define atributos	Interfaz para los materiales
MaterialImpreso	No define atributos	Producto de la modalidad presencial
LicenciaPlataform a	No define atributos	Producto de la modalidad virtual
Carne	No define atributos	Interfaz para los carnés
CarneFisico	No define atributos	Producto de la modalidad presencial
CarneDigital	No define atributos	Producto de la modalidad virtual

Enumeraciones

Enumeración	Valores definidos
EstadoProgram a	ACTIVO SUSPENDIDO

	FINALIZADO
Modalidad	PRESENCIAL VIRTUAL

Relación general de la agrupación

La estructura de clases puede resumirse de la siguiente manera:

- **Usuario** es una clase abstracta de la cual heredan **Estudiante** y **Docente**.
- **Programa** es una clase abstracta de la cual heredan **ProgramaBasico**, **ProgramaIntensivo** y **ProgramaPersonalizado**.
- **Matricula** relaciona un **Estudiante**, un **Programa**, un **Docente** opcional y varios **ServicioAdicional**.
- **Academia** administra las colecciones principales de estudiantes, docentes, programas, periodos, matrículas y servicios.
- **PeriodoAcademico** contiene una lista de programas ofertados.
- **ProgramaFactory** y sus fábricas concretas permiten crear los diferentes tipos de programas.
- **FabricaInsumos** permite crear familias compatibles de **Material** y **Carne** según la modalidad.
- **GeneradorComprobante** permite generar comprobantes mediante diferentes formatos.
- **ConsecutivoMatricula** centraliza la generación del número consecutivo de las matrículas.
- **Matricula.Builder** permite construir una matrícula mediante atributos obligatorios y opcionales.

¿Qué funcionalidades se solicitan finalmente?

Requisitos funcionales

RF-01: Registrar una matrícula con datos obligatorios y opcionales.

RF-02: Calcular si el teléfono de un estudiante es un número perfecto.

RF-03: Acumular el total de ingresos generados en un periodo específico.

RF-04: Clonar la oferta base para abrir un nuevo periodo académico.

RF-05: Emitir el comprobante de pago en PDF o Excel.

Reglas de negocio

RN-01: Toda matrícula debe tener estudiante, programa y fecha de inicio.

RN-02: El descuento de la matrícula no puede superar el 30% del valor del programa.

RN-03: El número de matrícula es consecutivo y no se repite nunca.

RN-04: Llenar cupos en un periodo no afecta los cupos del periodo siguiente.

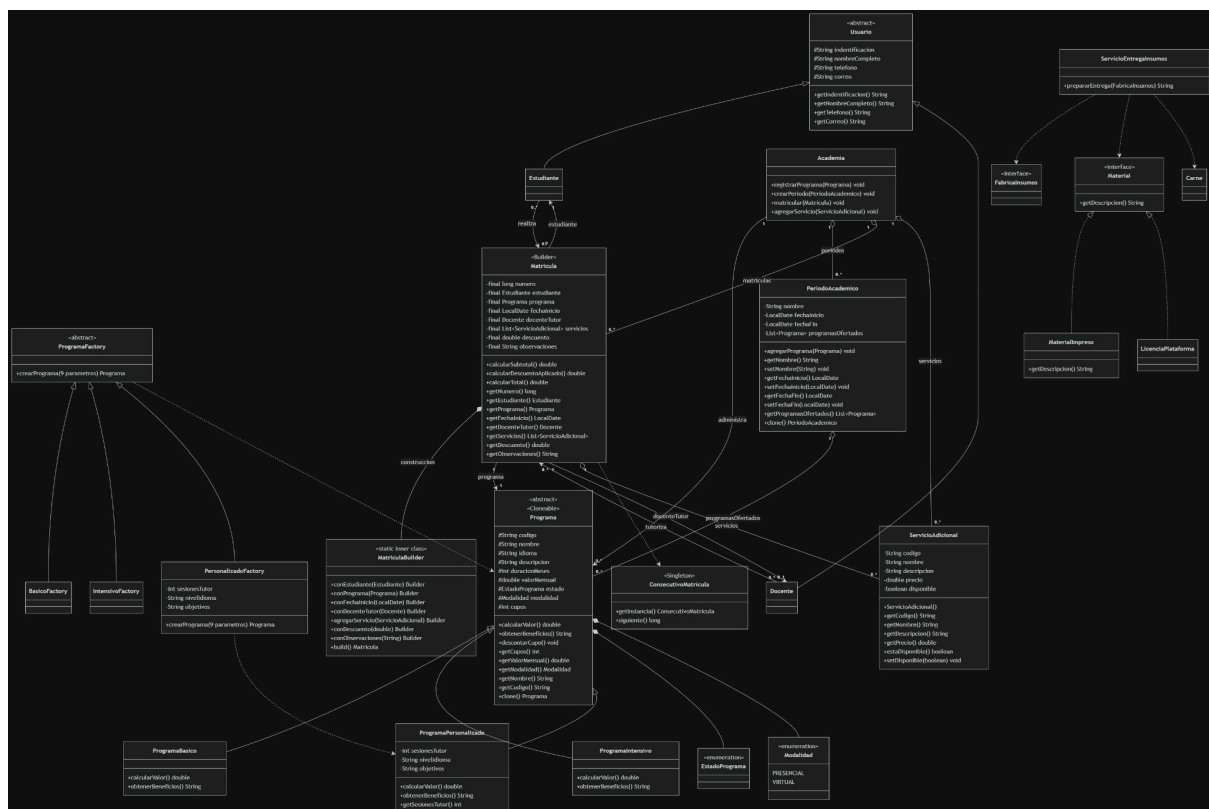
RN-05: Los insumos no se mezclan: material impreso va con carné físico, y licencia virtual con carné digital.

Descomposición:

¿Cómo se distribuyen las funcionalidades? UML

En el siguiente enlace se adjunta al archivo en drive con el diagrama para una mejor visualización:

https://drive.google.com/drive/folders/1QuPu7-Dq34BhA_n09EmTRQLhaxgG47dw?usp=drive_link



¿Qué debo hacer para probar las funcionalidades?

Reconocimiento de patrones:

¿Qué puedo reutilizar de la solución de otros problemas?

Arquitectura MVC

Patrón singleton para gestionar la generación del número de matrícula.

Patrón builder para la clase matrícula.

Patrón Prototype para abrir periodos académicos

Patrón Abstract factory para la creación de las familias de los insumos según la modalidad.

Patrón Factory (method) para la creación de los programas.

Patrón Factory (method) para la creación de los comprobantes.

Codificación:

¿Cómo pruebo la solución en java?

Matrícula

- Crear una matrícula con todos los datos obligatorios.
- Intentar crear una matrícula sin estudiante.
- Intentar crear una matrícula sin programa.
- Intentar crear una matrícula sin fecha de inicio.
- Crear una matrícula con descuento del 30%.
- Intentar crear una matrícula con descuento superior al 30%.

Consecutivo

- Crear varias matrículas.
- Comprobar que cada una recibe un número diferente y consecutivo.

Número perfecto

- Buscar un estudiante por su teléfono.
- Comprobar un teléfono cuyo número sea perfecto.
- Comprobar un teléfono que no sea un número perfecto.
- Comprobar un teléfono que no pueda convertirse correctamente a número.

Programas y cupos

- Crear programas básicos, intensivos y personalizados.
- Descontar cupos de un programa.
- Clonar un periodo académico.
- Modificar los cupos de un periodo y comprobar que los cupos del periodo clonado permanecen independientes.

Modalidades

- Crear una fábrica presencial y comprobar que genera material impreso y carné físico.
- Crear una fábrica virtual y comprobar que genera licencia de plataforma y carné digital.

Ingresos

- Registrar varias matrículas.
- Seleccionar un periodo académico.
- Calcular los ingresos.
- Comprobar que solamente se incluyen las matrículas cuya fecha de inicio pertenece al periodo seleccionado.

Comprobantes

- Generar un comprobante mediante GeneradorPDF.
- Generar un comprobante mediante GeneradorExcel.
- Comprobar que ambos formatos contienen la información de la matrícula y su valor total.

¿Cómo escribo la solución en java?

Enlace repositorio:

<https://github.com/marianavz02/Parcial1LinguaPlus/tree/main/src/main/java>